UNIVERSIDAD PANAMERICANA

Fundamentos de Programación en Paralelo

Profesor: Dr. Juan Carlos López Pimentel

Estadio Paralelo

Simulador distribuido de un estadio de fútbol

Evidencia: **3pf_FirstProjectProgress**
Primer avance del proyecto final: Análisis y Diseño

Integrantes

Heidi Meiners Muñoz
Valeria Pérez Maciel
Fátima Sofía Walls Fernández
Alberto Stephen Dubin Hernández

3 de noviembre de 2026

Índice

1. Descripción del proyecto **2**

2. Agentes y sus tareas **2**
2.1. Aficionado . . . 3
2.2. Guardia . . . 3
2.3. Cocinero . . . 3
2.4. Mesero . . . 4
2.5. Vendedor de gradas . . . 4
2.6. Reloj del partido . . . 4

3. Secciones críticas y zonas críticas **4**
3.1. Secciones críticas (búfer 1) . . . 5
3.2. Zonas críticas (búfer > 1) . . . 5
3.3. Prevención de deadlock . . . 6
3.4. Resumen de secciones críticas . . . 6

4. Contexto distribuido **6**
4.1. Migración de agentes . . . 7
4.2. Difusión de eventos . . . 7
4.3. Protocolo . . . 7

5. Estados de los agentes y estados de los hilos **8**

6. Diagramas BPMN **8**

1. Descripción del proyecto

Estadio Paralelo es un simulador gráfico, escrito en Java, de un estadio de fútbol durante un partido. Cientos de personas llegan, pasan por la entrada, buscan su lugar, ven el partido, van al baño, compran comida y finalmente se retiran, todo al mismo tiempo. En el simulador cada una de esas personas es un *agente*, es decir, un hilo de ejecución independiente con su propio comportamiento. Los agentes compiten entre sí por recursos limitados: torniquetes, cubículos de baño, porciones de comida, lugares en la escalera y asientos numerados.

El objetivo del proyecto es poner en práctica los mecanismos de sincronización vistos en el curso (candados, monitores y semáforos) en un escenario cotidiano donde la concurrencia aparece de manera natural. Sin estos mecanismos, dos aficionados podrían sentarse en el mismo asiento, el mostrador de comida podría quedar con porciones negativas o más personas de las permitidas podrían entrar a un baño. El simulador permite observar estas situaciones y la manera en que se evitan, incluida la prevención de *deadlock*.

El estadio está dividido en tres pisos y cada piso corre en una computadora distinta, de modo que el sistema funciona en un contexto **cliente-servidor distribuido**:

- **Planta baja (S1):** la cancha y las gradas bajas, con la única entrada del estadio, una tienda y un baño general.
- **Palcos (S2):** la zona exclusiva, donde cada palco tiene baño propio y los meseros llevan la comida.
- **Grada alta (S3):** la zona popular, con lugares más baratos, una tienda, un baño general y vendedores que recorren las gradas.

Todos los aficionados entran por la planta baja. Quien tiene boleto de palco o de grada alta sube por la escalera y, al llegar al final, su agente *migra* a la computadora del piso correspondiente. Al terminar el partido hace el recorrido inverso.

Un hilo especial, el *reloj del partido*, marca el inicio, los goles, el medio tiempo y el final, y difunde estos eventos a los tres servidores. Los eventos modifican el comportamiento de todos los agentes. El más interesante es el medio tiempo, en el que la mayoría de los aficionados decide ir al baño o a comprar comida al mismo tiempo, lo que provoca la mayor contención sobre los recursos compartidos.

Cada servidor muestra una ventana con el plano de su piso y los agentes moviéndose en tiempo real, una **tabla de estados** con el estado de cada agente (tanto el estado del dominio, por ejemplo “en fila del baño”, como el estado real del hilo en Java: `RUNNABLE`, `BLOCKED`, `WAITING`, etc.) y contadores de ocupación de cada recurso. El simulador incluye sonido (silbato, grito de gol y murmullo de la multitud) y, al iniciar, permite configurar cuántos agentes de cada tipo entran a la simulación y la capacidad de cada recurso.

2. Agentes y sus tareas

El sistema tiene seis tipos de agentes. Cada uno es un hilo (`Thread`) con una máquina de estados propia; un *estado* es un lugar o actividad donde el agente permanece un tiempo (por ejemplo, “viendo el partido”) y una *transición* es el paso de un estado a otro (por ejemplo, “caminando hacia el baño”). La Tabla 1 resume los agentes.

Agente	Piso	Interactúa con
Aficionado	S1 → S2/S3	Guardia, Cocinero (mostrador), Mesero, Vendedor de gradas
Guardia	S1, S2	Aficionado
Cocinero	S1, S2, S3	Aficionado, Mesero, Vendedor de gradas (a través del mostrador)
Mesero	S2	Aficionado de palco, Cocinero
Vendedor de gradas	S1, S3	Aficionado, Cocinero
Reloj del partido	S1	Todos los agentes (por medio de eventos)

Tabla 1: Tipos de agentes del sistema.

2.1 Aficionado

Es el agente principal y el único que cambia de computadora. Llega con un boleto de tipo *General*, *Palco* o *Popular* (algunos pueden ser falsos) y tiene niveles de hambre y de vejiga que aumentan con el tiempo.

- Camina a la entrada, se forma en la fila del torniquete y espera a que un guardia revise su boleto. Si es inválido, es expulsado.
- Según su boleto, va a su asiento en la planta baja o sube la escalera y migra al servidor de palcos o de grada alta.
- Busca y ocupa su asiento numerado y se queda viendo el partido.
- Cada cierto tiempo decide qué hacer: seguir viendo, ir al baño, ir a la tienda (pagar en caja y tomar una porción del mostrador), llamar al vendedor de gradas, pedir comida al mesero (solo en palcos) o salir. Cuando hay un gol, celebra.
- Al terminar el partido, baja la escalera, migra de regreso a la planta baja y sale del estadio.

2.2 Guardia

Trabaja en los torniquetes de la entrada (S1) y en el acceso a los palcos (S2).

- Espera en su puesto a que llegue un aficionado. Si pasa un tiempo sin que nadie llegue, pasa a su zona de decisión.
- Cuando llega un aficionado, revisa su boleto: si es válido y de la zona correcta le permite el paso; de lo contrario lo rechaza.
- En la zona de decisión elige seguir en su puesto, tomar un descanso o retirarse cuando termina el partido y ya no hay aficionados en la entrada.

2.3 Cocinero

Es el *productor* de la comida. Hay cocineros en las tiendas de S1 y S3 y en la cocina de palcos de S2.

- Prepara una porción de comida (actividad con duración).
- Coloca la porción en el mostrador. Si el mostrador está lleno, espera hasta que algún consumidor tome una porción.
- Al colocarla, notifica a los consumidores que esperaban comida.
- Decide si sigue cocinando, toma un descanso o se retira al final del partido.

2.4 Mesero

Atiende los pedidos de los palcos (S2).

- Espera pedidos en la cocina de palcos; cuando llega uno, lo toma de la cola de pedidos.
- Toma una bandeja, recoge la comida del mostrador de la cocina y sube en el montacargas al nivel de los palcos.
- Entrega el pedido al aficionado, le avisa que ya llegó su comida, baja, libera el montacargas y devuelve la bandeja.
- Decide si sigue atendiendo, descansa o se retira cuando termina el partido y no quedan pedidos.

2.5 Vendedor de gradas

Recorre las gradas de S1 y S3 vendiendo comida.

- Va a la tienda y llena su charola tomando porciones del mostrador (compite con los aficionados que compran ahí).
- Recorre las gradas esperando llamados de los aficionados. Cuando alguien lo llama, camina a su asiento y le vende.
- Si la charola se vacía regresa a la tienda a recargar; si no, sigue recorriendo las gradas o descansa.

2.6 Reloj del partido

Es un agente que no se desplaza, pero coordina a todos los demás.

- Espera a que el estadio se llene y emite el evento de **inicio**.
- Simula el partido minuto a minuto; en cada minuto puede ocurrir un **gol** de forma aleatoria.
- Al terminar el primer tiempo emite el **medio tiempo**, espera el descanso y comienza el segundo tiempo.
- Al terminar emite el **fin del partido**. Cada evento se difunde a los tres servidores.

3. Secciones críticas y zonas críticas

Llamamos **sección crítica (búfer 1)** a un recurso que solo puede ser usado por un agente a la vez, y **zona crítica (búfer $>$ 1)** a un recurso con varias unidades que pueden usarse simultáneamente hasta una capacidad máxima. A cada búfer se le asignó una letra, que es la misma que aparece en los diagramas BPMN (con apóstrofo, por ejemplo `T'`, cuando el recurso se libera).

Letra	Búfer	Piso	Capacidad	Mecanismo en Java
`Bp`	Baño de palco	S2	1	`ReentrantLock` / `synchronized`
`Mc`	Montacargas	S2	1	`ReentrantLock`
`A`	Asiento numerado	S1, S3	1 por asiento	`synchronized` sobre cada asiento
`T`	Torniquetes	S1	T	`Semaphore(T, true)`
`Bg`	Baño general (cubículos)	S1, S3	N	`Semaphore(N, true)`
`Cj`	Cajas de cobro	S1, S3	C	`Semaphore(C)`
`Bj`	Bandejas de meseros	S2	B	`Semaphore(B)`
`E`	Escalera (un sentido a la vez)	S1↔S2/S3	E	Monitor: `ReentrantLock` + `Condition`
`M`	Mostrador de comida	S1, S2, S3	K porciones	Monitor: `synchronized` + `wait()/notifyAll()`
`Pd`	Cola de pedidos de palco	S2	P	Monitor (productor-consumidor)
`Ll`	Cola de llamados al vendedor	S1, S3	L	Monitor (productor-consumidor)

Tabla 2: Búferes del sistema. Las capacidades T, N, C, B, E, K, P, L se configuran al iniciar.

3.1 Secciones críticas (búfer 1)

- **Baño de palco (`Bp`).** Cada palco tiene un solo baño. El aficionado adquiere el candado del baño de su palco; si otro aficionado lo está usando, queda bloqueado hasta que el primero lo libere. Esto garantiza exclusión mutua.
- **Montacargas (`Mc`).** Solo un mesero puede usarlo a la vez para subir los pedidos al nivel de los palcos.
- **Asientos numerados (`A`).** Cada asiento es un objeto con estado libre u ocupado. Ocuparlo es un método `synchronized`: si dos aficionados con el mismo número de asiento (boleto duplicado) llegan al mismo tiempo, solo uno lo consigue y el otro es atendido como un caso de boleto inválido.

3.2 Zonas críticas (búfer > 1)

- **Torniquetes (`T`).** Un semáforo con T permisos limita cuántos aficionados pueden estar en revisión al mismo tiempo. Dentro del torniquete ocurre una *cita* entre el aficionado y el guardia: el aficionado avisa que llegó y espera el veredicto; el guardia espera a que llegue un aficionado, revisa el boleto y le comunica el resultado. Esta cita se implementa con un par de semáforos.
- **Baños generales (`Bg`).** Un semáforo justo (*fair*) con N permisos, uno por cubículo. Quien llega cuando todos están ocupados espera su turno en orden de llegada.
- **Cajas (`Cj`) y bandejas (`Bj`).** Semáforos contadores con C y B permisos respectivamente.
- **Escalera (`E`).** Admite hasta E personas, pero solo en un sentido a la vez (como un puente de un carril). Se implementa como monitor con un candado y dos condiciones (`puedeSubir`, `puedeBajar`). Para evitar inanición, después de que cierto número de personas pasa en un sentido se da preferencia al sentido contrario.
- **Mostrador de comida (`M`).** Es el problema clásico de **productor-consumidor** con búfer acotado de K porciones. Los cocineros producen; los aficionados, los vendedores de gradas y los meseros consumen. Se implementa como un monitor propio: si el mostrador está lleno el

cocinero hace `wait()`, si está vacío el consumidor hace `wait()`, y cada operación termina con `notifyAll()`.

- **Cola de pedidos (`Pd`) y cola de llamados (`Ll`).** También son productor-consumidor: los aficionados producen pedidos o llamados y los meseros o vendedores los consumen. Así se garantiza que cada pedido se atienda exactamente una vez.

3.3 Prevención de deadlock

El mesero es el único agente que necesita dos recursos a la vez: una bandeja (`Bj`) y el montacargas (`Mc`). Si un mesero tomara primero el montacargas y otro primero la bandeja, podrían quedar esperándose mutuamente para siempre (espera circular). Para evitarlo, **todos los meseros adquieren los recursos en el mismo orden**: primero la bandeja y después el montacargas, y los liberan en orden inverso. Al eliminar la espera circular, el deadlock no puede ocurrir. El simulador tendrá un modo de demostración que invierte el orden en la mitad de los meseros para mostrar el problema y compararlo con la solución.

3.4 Resumen de secciones críticas

#	Sección crítica	Qué pasaría sin protección
1	Entrar a un torniquete	Más de T personas en revisión a la vez
2	Revisar un boleto (cita)	Dos guardias revisan al mismo aficionado o uno revisa a dos a la vez
3	Subir o bajar la escalera	Personas en ambos sentidos; más de E en la escalera
4	Ocupar un asiento	Dos aficionados en el mismo lugar
5	Usar el baño general	Más de N personas en el baño
6	Usar el baño del palco	Dos personas en el mismo baño
7	Poner o tomar del mostrador	Porciones negativas, más de K, o comida duplicada
8	Pagar en caja	Más de C cobros simultáneos
9	Encolar o tomar un pedido	Pedidos perdidos o entregados dos veces
10	Encolar o tomar un llamado	Dos vendedores atienden al mismo aficionado
11	Usar bandeja y montacargas	Deadlock entre meseros
12	Actualizar contadores de la interfaz	Contadores incorrectos o errores al dibujar

Tabla 3: Secciones críticas identificadas.

4. Contexto distribuido

El sistema se ejecuta en **tres computadoras**, una por piso del estadio (Figura 1). Las tres corren el mismo programa y se distinguen por los argumentos de inicio (`-piso`, `-puerto` y `-vecinos`). Cada servidor administra sus propios recursos y agentes, tiene su propia ventana gráfica y su propia tabla de estados, y se comunica con los demás mediante **sockets TCP**.

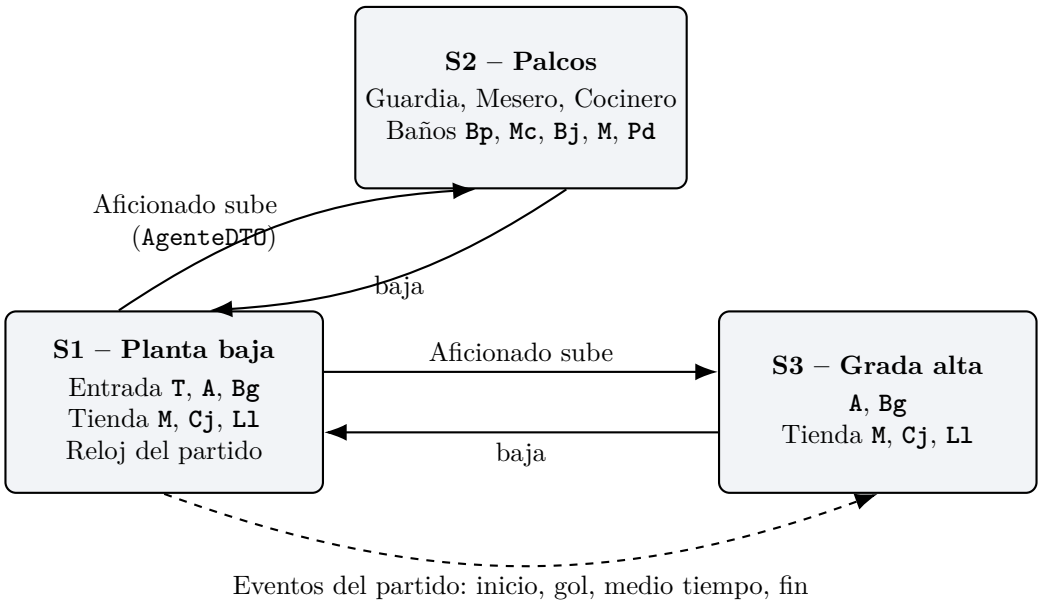


Figura 1: Distribución del estadio en tres servidores. Las flechas continuas son migraciones de agentes; la punteada es la difusión de eventos (también llega a S2).

4.1 Migración de agentes

El agente que cambia de computadora es el **aficionado**. La entrada al estadio solo existe en la planta baja, por lo que todos los aficionados nacen en S1. Cuando un aficionado con boleto de palco o de grada alta termina de subir la escalera:

1. Su hilo guarda su estado en un objeto serializable `AgenteDTO` (identificador, tipo de boleto, número de asiento, niveles de hambre y vejiga, estado actual).
2. Abre una conexión con el servidor destino y envía el objeto con un `ObjectOutputStream`.
3. El hilo local termina (estado `TERMINATED`) y el aficionado desaparece de la pantalla de S1.
4. En el servidor destino, un hilo dedicado (`ServidorRed`) espera conexiones con `accept()`. Al recibir el objeto, crea un nuevo hilo de aficionado con ese estado y lo inicia; el aficionado aparece en la pantalla del nuevo piso y continúa su comportamiento.

Al terminar el partido, el aficionado baja la escalera y migra de la misma forma de regreso a S1 para salir del estadio. En S2, al llegar, pasa además por el guardia del acceso a palcos.

4.2 Difusión de eventos

El reloj del partido vive en S1. Cada vez que ocurre un evento (inicio, gol, medio tiempo o fin), S1 envía un mensaje de tipo `EVENTO` a S2 y S3. El hilo `ServidorRed` de cada piso lo recibe, notifica a sus agentes locales y reproduce el sonido correspondiente. Así, por ejemplo, en el medio tiempo los aficionados de los tres pisos reaccionan al mismo tiempo.

4.3 Protocolo

Todos los mensajes entre servidores son objetos `Mensaje` con dos tipos: `AGENTE`, que lleva un `AgenteDTO`, y `EVENTO`, que lleva el tipo de evento y el marcador. Para validar el sistema se comprobará

que la suma de aficionados en los tres pisos sea siempre igual al número de aficionados que entraron menos los que salieron, es decir, que ningún agente se pierda al migrar.

5. Estados de los agentes y estados de los hilos

Es importante distinguir dos conceptos:

- El **estado del agente** describe lo que la persona está haciendo en el estadio (por ejemplo, *viendo el partido*, *en fila del baño*, *comiendo*). Estos estados y sus transiciones son los que se modelan en los diagramas BPMN.
- El **estado del hilo** es el que reporta la máquina virtual de Java mediante `Thread.getState()`.

La tabla de estados del simulador mostrará ambos para cada agente, de forma que se pueda ver, por ejemplo, que un aficionado *en fila del baño* tiene su hilo en estado `WAITING`.

Estado del hilo	Cuándo se observa en el simulador
`NEW`	Agente creado que aún no inicia, incluido el que se acaba de recrear tras migrar
`RUNNABLE`	Caminando, preparando comida, revisando un boleto
`BLOCKED`	Esperando entrar a un método `synchronized` (asiento, mostrador)
`WAITING`	En `wait()` del mostrador, `acquire()` de un semáforo o `await()` de la escalera
`TIMED_WAITING`	Viendo el partido, usando el baño o descansando (`sleep`); esperas con límite de tiempo
`TERMINATED`	Agente que salió del estadio o que migró a otro servidor

Tabla 4: Estados de un hilo en Java y su significado en el simulador.

6. Diagramas BPMN

Los diagramas BPMN de cada agente se entregan en un documento aparte (*Diagramas BPMN – Estadio Paralelo*), con un diagrama por página para que sean legibles. Se incluyen:

1. Sistema general (programa principal de cada servidor).
2. Aficionado.
3. Guardia.
4. Cocinero.
5. Mesero.
6. Vendedor de gradas.
7. Reloj del partido.
8. Hilo de red y migración de agentes.

En los diagramas se marcan los búferes con la letra de la Tabla 2. Los pasos de migración (“Migrar a S2/S3” y “Migrar a S1”) y el diagrama del hilo de red muestran el contexto distribuido.